AIMERZ.ai
Aim.Act.Achieve

NOTES

Fundamentals of Java - 5

The System Class in Java

In Java, the `System` class is a part of the `java.lang` package and provides a collection of useful methods and fields. It is primarily used for accessing system resources, interacting with the environment, and performing input and output tasks.

Main Features of the `System` Class are**:**

Standard Input, Output, and Error Streams

Fields: `System.in`, `System.out`, `System.err`

Explanation: The `System` class has three fields – `in`, `out`, and `err` – which represent standard input, output, and error streams. These fields help in reading data from the keyboard, displaying data on the screen, and printing error messages.

Real-World Analogy: Imagine `System.in` as a microphone that lets you input data (like speaking to a device), `System.out` as a speaker that lets the device talk back, and `System.err` as an alarm or buzzer that only sounds when there's a problem.

Usage:

> `System.in`: Used with `Scanner` to read user input.
>
> `System.out`: Used with `println()` to display output on the console.
>
> `System.err`: Used to print error messages.

Examples:

```java
import java.util.Scanner;

public class SystemExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter your name:");
        String name = scanner.nextLine();
        System.out.println("Hello, " + name);
```

```
        // Error example
        System.err.println("This is an error message.");
    }
}
```

Environment Variables

Method: `System.getenv()`

Explanation: The `System.getenv()` method fetches environment variables, which are special settings that configure how the system behaves. These variables might store information about the system's setup, such as the file path or username.

Real-World Analogy: Imagine environment variables as a set of instructions or "house rules" that tell applications where to find essentials (like your documents or programs) or the name of the system user.

Usage:

Fetching specific environment variables like `JAVA_HOME` or `PATH`.

Example:

```Java
public class EnvironmentExample {
    public static void main(String[] args) {
        // Fetch JAVA_HOME environment variable
        String javaHome = System.getenv("JAVA_HOME");
        System.out.println("JAVA_HOME: " + javaHome);

        // Print all environment variables
        System.out.println("All Environment Variables:");
        System.getenv().forEach((key, value) -> System.out.println(key + ":
" + value));
    }
}
```

Current Time in Milliseconds and Nanoseconds

Methods: `System.currentTimeMillis()`, `System.nanoTime()`

Explanation: These methods provide the current time. `currentTimeMillis()` gives the time in milliseconds since January 1, 1970 (the "epoch"), while `nanoTime()` provides the time in nanoseconds.

Real-World Analogy: Imagine you have a stopwatch with two modes. `currentTimeMillis()` is like checking the date and time on your phone, and `nanoTime()` is like a high-speed stopwatch measuring every tiny fraction of a second.

Usage: Measuring how long it takes for code to run.

Examples:

```java
public class TimeExample {
    public static void main(String[] args) {
        // Getting the current time in milliseconds
        long startTime = System.currentTimeMillis();

        // Code you want to time
        for (int i = 0; i < 100; i++) {
            System.out.print(".");
        }

        long endTime = System.currentTimeMillis();
        System.out.println("\nExecution Time: " + (endTime - startTime) + "
ms");
    }
}
```

Output

```java
Execution Time: 4 ms
```

Array Copying

Method: `System.arraycopy()`

Explanation: The `System.arraycopy()` method allows for efficient copying of elements from one array to another.

Real-World Analogy: Imagine copying items from one shelf to another. Instead of moving each item one by one, `arraycopy()` quickly transfers the whole batch from one shelf to another.

Usage: When we need to move or clone elements from one array to another.

Example:

```Java
public class ArrayCopyExample {
    public static void main(String[] args) {
        int[] sourceArray = {1, 2, 3, 4, 5};
        int[] destinationArray = new int[5];

        // Copying array elements
        System.arraycopy(sourceArray, 0, destinationArray, 0,
sourceArray.length);

        // Displaying copied array
        for (int value : destinationArray) {
            System.out.print(value + " ");
        }
    }
}
```

System Properties

Method: `System.getProperty()`

Explanation: This method retrieves properties of the system, like the OS name, Java version, and file separator.

Real-World Analogy: Think of system properties as a card containing details about a device, like its brand, operating system, or installed software.

Usage: Checking the Java version or OS details for compatibility.

Example:

```Java
public class PropertiesExample {
    public static void main(String[] args) {
        String osName = System.getProperty("os.name");
        String javaVersion = System.getProperty("java.version");
        String userHome = System.getProperty("user.home");

        System.out.println("Operating System: " + osName);
        System.out.println("Java Version: " + javaVersion);
        System.out.println("User Home Directory: " + userHome);
    }
}
```

Garbage Collection

Method: `System.gc()`

Explanation: The `System.gc()` method is a suggestion to the JVM (Java Virtual Machine) to run the garbage collector, which helps clean up unused objects from memory.

Real-World Analogy: Imagine you have a cleaning robot at home that clears out unused items. By calling `System.gc()`, you're asking the robot to start cleaning.

Usage: Freeing memory by cleaning up unneeded objects. However, it is a request, not a guarantee.

Example:

```java
public class GarbageCollectionExample {
    public static void main(String[] args) {
        System.out.println("Requesting garbage collection...");
        System.gc(); // Suggest JVM to start garbage collection
        System.out.println("Garbage collection requested.");
    }
}
```

Exiting the Program

Method: `System.exit(int status)`

Explanation: The `System.exit()` method terminates the running Java application. The argument specifies the exit status; `0` typically means the program exited successfully, and any other number usually indicates an error.

Real-World Analogy: Imagine pressing a "Stop" button on a machine, where `0` means a normal stop and other numbers indicate specific issues or errors.

Usage:

> Useful when you need to close the program at a specific point or if an unrecoverable error occurs.

Example:

```java
public class ExitExample {

    public static void main(String[] args) {

        System.out.println("Program is running...");


        // Stop program with exit code 0 (normal termination)

        System.exit(0);
```

```
        // This code will never be reached
        System.out.println("This won't print.");
    }
}
```

Understanding Scanner Class in Java

What is a Scanner?

Scanner is our friend for reading input in Java!

Scanner is like a helper tool in Java that reads (takes in) information. Think of it like a person who reads:

- What we type on keyboard
- Text from files
- Numbers
- Words
- Pretty much anything we want to give to our program!

Conceptual Explanation: The Scanner class in Java is a powerful and flexible way to read input from various sources, such as the keyboard, files, and strings. It is part of the java.util package and was introduced in Java 5 to make reading different types of input (like numbers, strings, etc.) easier and more efficient for developers.

Why the Scanner Class Was Introduced

Before Scanner, reading input in Java required more complicated steps using classes like BufferedReader, which could be less intuitive for beginners and required additional conversion for reading different data types. Scanner simplified input handling by allowing direct reading of various types of data (integers, strings, doubles, etc.) in a user-friendly manner.

If we see it simply, Why Do We Need Scanner ?

- To make programs that can talk with users
- To read information from files
- To break down text into smaller pieces
- To convert text into numbers easily

How the Scanner Class Works

The Scanner class uses *tokens* to break down input into manageable parts. A token can be a word, a number, or another simple data unit. For example, in a sentence, each word can be a token.

Real World Analogy: Think of the Scanner class as a cashier in a grocery store. The cashier scans each item (or token) one at a time and puts it aside, ready to handle the next one. Similarly, Scanner scans input piece by piece, making it easier to work with each part.

How to Start Using Scanner

First Step: Import Scanner: import java.util.Scanner;

This is like telling Java "Hey Java! I want to use the Scanner tool in my program"

Second Step: Create Scanner

```Java
Scanner input = new Scanner(System.in); // Scanner Object Created
```

Different Constructors of the `Scanner` Class in Java

The `Scanner` class in Java provides multiple constructors to handle input from various sources. This flexibility allows it to read data from files, strings, input streams, and more.

public Scanner(Readable source)

This constructor creates a Scanner that reads from any Readable source, like a StringReader. Readable is an interface that can read a sequence of characters.

Real-World Analogy: Imagine reading from a book. The book (the source) provides content in a format that can be easily read.

Example:

```
Java
Readable readableSource = new java.io.StringReader("Hello, world!");
Scanner scanner = new Scanner(readableSource);
System.out.println(scanner.nextLine()); // Output: Hello, world!
```

Use Case: This is useful for testing when you need to simulate an input source.

public Scanner(InputStream source)

This constructor reads input from an InputStream, such as System.in (keyboard input) or a network socket. InputStream is commonly used to read data byte by byte.

Real-World Analogy: Think of a microphone receiving sound from a user. Here, the InputStream is like a channel that captures incoming data, and Scanner interprets it.

Example:

```
Java
Scanner scanner = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = scanner.nextLine();
System.out.println("Hello, " + name + "!");
```

Use Case: This is the most common way to use `Scanner` to read user input from the keyboard.

public Scanner(InputStream source, String charsetName)

This constructor reads input from an InputStream using a specified character set (like "UTF-8" or "ISO-8859-1"). The character set tells Scanner how to interpret the bytes as text.

Real-World Analogy: Imagine translating a book written in another language. The charset here is like a language translator, helping interpret data in the correct format.

Example:

```
Java
Scanner scanner = new Scanner(System.in, "UTF-8");
System.out.print("Enter a message: ");
String message = scanner.nextLine();
System.out.println("Message: " + message);
```

For example run the program and enter below words in different languages

`Hello, world!`
こんにちは (Japanese)
`Café` (French with accented "é")
`¡Hola! ¿Cómo estás?` (Spanish with inverted punctuation)
दिल्ली के मुख्यमंत्री (In Hindi)

Output will be

```
Java
Enter a message: こんにちは, welcome to Café!
```

Use Case: This is useful when you're reading input in a specific encoding, especially with non-English characters.

public Scanner(InputStream source, Charset charset)

This constructor also uses an InputStream but with a Charset object instead of a String. The Charset object is more flexible for handling encodings.

Example:

```
Java
import java.nio.charset.StandardCharsets;
Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8);
System.out.print("Enter a message: ");
String message = scanner.nextLine();
System.out.println("Message: " + message);
```

For example run the program and enter below words in different languages

`Hello, world!`
`こんにちは` (Japanese)
`Café` (French with accented "é")
`¡Hola! ¿Cómo estás?` (Spanish with inverted punctuation)
`दिल्ली के मुख्यमंत्री` (In Hindi)

Output will be

```
Java
Enter a message: こんにちは, welcome to Café!
```

Use Case: Similar to the previous constructor, it's helpful for handling different character encodings.

public Scanner(File source) throws FileNotFoundException

This constructor allows the Scanner to read data from a File. It will throw a FileNotFoundException if the file doesn't exist.

Real-World Analogy: Imagine a librarian who tries to read a book from the library but can't find it on the shelf if it's missing.

Example:

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FileReaderExample {
    public static void main(String[] args) throws FileNotFoundException {
        // We create a File object for "data.txt" to refer to the file we want to read
        File file = new File("data.txt");

        // We use a Scanner object with the File object to read the file's content
        Scanner scanner = new Scanner(file);

        // Loop to read and print each line from the file until no lines are left
        while (scanner.hasNextLine()) {
            System.out.println(scanner.nextLine());
        }
    }
}
```

Use Case: It's helpful for reading files that use a particular character encoding, such as files with special symbols or international characters.

Explanation of Scanner and File objects and Inbuilt methods

File object: We create a File object (new File("data.txt")) to represent the file data.txt. This object serves as a reference to the file and helps Scanner locate it on the system.

Scanner object: We use a Scanner object with the File object to read the file line by line. Scanner makes reading files easy because it has built-in methods like hasNextLine() (to check if there's another line) and nextLine() (to read each line of text).

throws FileNotFoundException: Instead of handling the FileNotFoundException inside a try-catch block, we declare it with throws in the method header. This means that if data.txt isn't found, the program will throw an error and stop.

hasNextLine() and nextLine(): hasNextLine() checks if there are more lines, and nextLine() reads each line until the end of the file.

public Scanner(File source, String charsetName) throws FileNotFoundException

This constructor reads from a File with a specified charset, like "UTF-8". If the file doesn't exist, it throws a FileNotFoundException.

Example:

```Java
import java.io.File;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class FileReaderExample {
    public static void main(String[] args) throws IOException {
        // Create a File object for "data.txt" to reference the file we want
to read
        File file = new File("data.txt");

        // Use a Scanner with the File object and specify UTF-8 encoding to
read text in UTF-8
        Scanner scanner = new Scanner(file, StandardCharsets.UTF_8);

        // Loop to read and print each line from the file until no lines are
left
        while (scanner.hasNextLine()) {
            System.out.println(scanner.nextLine());
        }
    }
}
```

Explanation

Purpose: This code reads each line from a file named data.txt and prints it to the console using UTF-8 encoding.

File object: We create a File object (new File("data.txt")) to point to data.txt. This object is a reference that helps Scanner locate the file on the system.

Scanner object with UTF-8: We create a Scanner object with two parameters — the File object and StandardCharsets.UTF_8 — to specify that the file content should be read in UTF-8 encoding. This is important when the file may contain non-English characters or special symbols.

UTF-8 Encoding: Specifying StandardCharsets.UTF_8 ensures that the file is read with UTF-8 encoding, which handles various characters correctly, like accented letters or symbols.

throws IOException: Instead of handling potential errors (like file not found or unable to read) with a try-catch block, we declare throws IOException in the method signature. This allows the program to throw an IOException if any error occurs, stopping execution and letting the caller (or JVM) handle it.

public Scanner(File source, Charset charset) throws IOException

This constructor is similar to the previous one but uses a Charset object. It throws an IOException if the file can't be read.

Example:

```Java
import java.io.File;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class FileReaderExample {
    public static void main(String[] args) throws IOException {
        // Create a File object for "data.txt" to reference the file we want
to read
        File file = new File("data.txt");

        // Use a Scanner with the File object and specify UTF-8 encoding to
read text in UTF-8
        Scanner scanner = new Scanner(file, StandardCharsets.UTF_8);

        // Loop to read and print each line from the file until no lines are
left
        while (scanner.hasNextLine()) {
            System.out.println(scanner.nextLine());
        }
    }
}
```

Use Case: Like the previous, but allows for more precise handling with the Charset object.

Explanation

Purpose: This code reads and prints each line from the data.txt file, interpreting the text with UTF-8 encoding.

File object: File file = new File("data.txt"); creates a File object that represents the data.txt file on your system. This File object is necessary to help Scanner locate and access the file.

Scanner object with UTF-8 encoding: We create a Scanner object (new Scanner(file, StandardCharsets.UTF_8)) to read from the File object using UTF-8 encoding. Specifying StandardCharsets.UTF_8 is important when working with text that may contain special or non-English characters.

UTF-8 Encoding: By using StandardCharsets.UTF_8, we ensure that the text is read in UTF-8, which is the standard for handling a wide range of characters.

throws IOException: Instead of using a try-catch block, throws IOException is added to the method signature. This means if there's an issue, like the file not being found or unreadable, an IOException will be thrown, and the program will stop, leaving error handling to the caller or the JVM.

public Scanner(String source)

This constructor creates a Scanner that reads from a String source. This is helpful for parsing specific data from a text string.

Real-World Analogy: Think of a barcode reader reading individual items in a packed line of products.

Example:

```
Java
String data = "apple orange banana";
```

```
Scanner scanner = new Scanner(data);
while (scanner.hasNext()) {
    System.out.println(scanner.next());
}
```

Output

```
Java
apple
orange
banana
```

Use Case: Useful when you want to break down a String into tokens without external input sources.

Old

Common Scanner Methods (Tools) for Reading Different Types of Input

Creating Object:

Scanner input = new Scanner(System.in);

// Reading whole line of text

```
Java

String fullLine = input.nextLine();     // Reads "My name is John"
```

// Reading single word

```
Java

String word = input.next();             // Reads just "My"
```

// Reading a number

```
Java

int number = input.nextInt();           // Reads "123"
```

// Reading decimal number

```java
double decimal = input.nextDouble();     // Reads "12.34"
```

// Reading true/false

```java
boolean yesNo = input.nextBoolean();    // Reads "true"
```

Real Examples with Explanations

Example 1: Getting User's Information

```java
Scanner input = new Scanner(System.in);

System.out.println("What is your name?");

String name = input.nextLine();        // Reads full name like "Saqib"

System.out.println("How old are you?");

int age = input.nextInt();             // Reads age like "25"

System.out.println("How tall are you (in metres)?");

double height = input.nextDouble();     // Reads height like "1.75"
```

Important Rules to Remember

The nextLine() Problem

When using nextLine() after other methods like nextInt(), add an extra nextLine() otherwise it won't take input from the console

Example:

```Java
Scanner input = new Scanner(System.in);


System.out.println("Enter your age:");

int age = input.nextInt();

input.nextLine();  // Add this extra line!



System.out.println("Enter your name:");

String name = input.nextLine();
```

Always Close Scanner

If you don't close the Scanner object after using it, it can lead to a **resource leak**. This happens because Scanner uses resources like memory and file handlers to read input, and these need to be freed up when you're done.

Here's a simple **analogy**: Imagine you open a water faucet to fill a glass, but after filling the glass, you forget to turn it off. The water keeps flowing and gets wasted. Similarly, when you don't close a Scanner, the program keeps holding on to resources it no longer needs, which can waste memory and processing power.

```java
Scanner input = new Scanner(System.in);

// Use scanner here

input.close();  // Close when done!
```

Avoid Common Problems

Problem 1: Wrong Input Type

```java
System.out.println("Enter a number:");
int number = input.nextInt();
```

In above code if we enter any different value like String or double then we'll input exception

Problem 2: Checking if More Input Exists

```java
while(input.hasNext()) {

    String word = input.next();

    // Process word

}
```

In above code, the scanner will always open and asks for input, so be careful white working with such type of codes

Useful Scanner Methods List

Method	What It Does	Example Input
nextLine()	Reads whole line	"Hello World"
next()	Reads one word	"Hello"

nextInt( )	Reads whole number	"123"
nextDouble()	Reads decimal number	"12.34"
nextBoolean()	Reads true/false	"true"
hasNext()	Checks if more input exists	-
hasNextInt()	Checks if next input is number	-
hasNextLine()	Checks if more lines exist	-

Practice Examples

Example 1: Simple Name and Age Program

```java
Scanner input = new Scanner(System.in);
System.out.println("Enter your name:");
String name = input.nextLine();

System.out.println("Enter your age:");
int age = input.nextInt();

System.out.println("Hello " + name + "! You are " + age + " years old.");
input.close();
```

Suggestions for Using Scanner class

1. Always import Scanner at the start
2. Close Scanner when done
3. Use try-catch for safer input handling
   // Covered in Exception Handling topic
4. Remember the nextLine() issue after number inputs

5. Use clear prompts to tell users what to enter
6. Check if input exists before reading it

OutputStream : Understanding System.out

What is System.out?

System.out is Java's way of showing information to users. Think of it like:

- A messenger that shows messages on our computer screen
- A way to print information for users to see
- The most basic way to display output in Java

Basic Parts of System.out.println()

System→ The overall system class
.out → The output stream (where messages go)
.println() → The method to print something

Think of it like this:
- System is like our computer
- out is like our screen
- println() is like writing on that screen

Different Ways to Print by using inbuilt methods

println() - Print and Move to New Line

Prints and moves to next line

```java
System.out.println("Hello");
System.out.println("World");
```

Output:
Hello
World

print() - Print Without New Line

Prints on same line

```
Java
System.out.print("Hello ");
System.out.print("World");

//Output:Hello World
```

printf() - Print with Formatting
In Java, `printf()` is used to print text with specific formatting, like setting the number of decimal places or aligning text.

Prints with special formatting

```
Java
String name = "John";
int age = 25;
System.out.printf("Name: %10s, Age: %10d", name, age);
```

Output:
Name: John, Age: 25

Common Uses with Examples

Printing Simple Messages

```
Java
System.out.println("Welcome to my program!");
System.out.println("-------------------");
```

Printing Variables

```java
String name = "Saqib";
int age = 30;
double height = 1.65;

System.out.println("Name: " + name);
System.out.println("Age: " + age);
System.out.println("Height: " + height + " meters");
```

Printing Numbers

```java
int num1 = 10;
int num2 = 20;
int sum = num1 + num2;

System.out.println("First number: " + num1);
System.out.println("Second number: " + num2);
System.out.println("Sum: " + sum);
```

Special printf() Formatting

Common Format Specifiers

```java
%s  → For Strings
%d  → For integers
%f  → For decimals
%b  → For booleans
%n  → For new line
```

Printf Examples

```java
String name = "Tom";
double price = 19.99;
int quantity = 5;

// Basic formatting
System.out.printf("Customer %s bought %d items%n", name, quantity);

// Number formatting
System.out.printf("Price: $%.2f%n", price);  // Shows 2 decimal places

// Width formatting
System.out.printf("Name: %-10s Age: %d%n", name, 25);  // Left-aligned, 10
spaces
```

Real-World Examples

Simple Receipt Printer

```java
String item = "Coffee";
double price = 3.50;
int quantity = 2;

System.out.println("=== RECEIPT ===");
System.out.printf("Item: %s%n", item);
System.out.printf("Price: $%.2f%n", price);
System.out.printf("Quantity: %d%n", quantity);
System.out.printf("Total: $%.2f%n", price * quantity);
System.out.println("=============");

Output:
=== RECEIPT ===
Item: Coffee
Price: $3.50
Quantity: 2
Total: $7.00
=============
```

Progress Display

```java
System.out.print("Loading");
System.out.print(".");
System.out.print(".");
System.out.println(".");

Output:
Loading...
```

Common Patterns and Suggestions

Making Tables

```java
System.out.println("Name\tAge\tCity");
System.out.println("----\t---\t----");
System.out.println("John\t25\tNYC");
System.out.println("Mary\t30\tLA");

Output:
Name    Age     City
----    ---     ----
John    25      NYC
Mary    30      LA
```

Special Characters

```java
\n  → New line
\t  → Tab space
\"  → Double quote
\\  → Backslash
```

Common Mistakes to Avoid

Forgetting + for Concatenation

// Wrong

```java
System.out.println("Age: " age);
```

// Correct

```java
System.out.println("Age: " + age);
```

Wrong Format Specifiers

// Wrong

```java
System.out.printf("Price: %d", 19.99);  // %d is for integers!
```

// Correct

```java
System.out.printf("Price: %.2f", 19.99);
```

Practice Examples

Formatted Table

```java
String[] names = {"John", "Mary", "Bob"};
int[] ages = {25, 30, 35};

System.out.printf("%-10s %s%n", "Name", "Age");
System.out.printf("%-10s %s%n", "----", "---");

for(int i = 0; i < names.length; i++) {
    System.out.printf("%-10s %d%n", names[i], ages[i]);
}
```

Suggestions for Clean Output

1. Use println() when we want each piece on a new line
2. Use print() when we want everything on the same line
3. Use printf() when we need specific formatting
4. Add empty println() for blank lines
5. Use formatting for numbers to look clean
6. Test your output with different data sizes

Remember: Good output makes our program easy to understand

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan